1. Energy Conservation Before-and-After Analysis

Compute energy consumption changes across appliances, evaluate percentage savings, and assess efficiency goals.

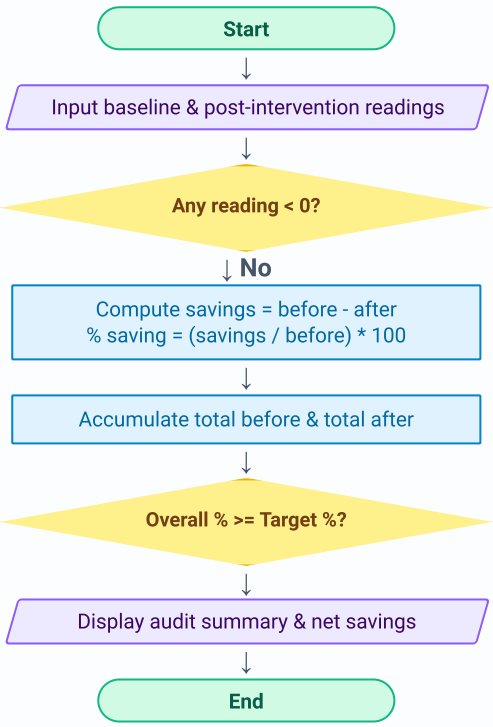
PROBLEM STATEMENT

> Develop a program to analyze energy consumption across multiple domestic appliances before and after implementing conservation measures. The program must read baseline usage (kWh) and post-intervention usage, calculate absolute reductions and percentage savings per appliance, ensure no recorded readings are negative, detect any unexpected surge in consumption, and determine whether the overall facility target saving threshold has been achieved.

PYTHON CODE

```
appliances = [ {"name": "HVAC Unit", "before": 450, "after": 360}, {"name": "Lighting", "before": 120, "after": 75}, {"name": "Refrigeration", "before": 200, "after": 190}, {"name": "Water Heater", "before": 180, "after": 125} ] target_saving_pct = 20.0 total_before, total_after = 0, 0 print("--- APPLIANCE ENERGY AUDIT ---") for app in appliances: b, a = app["before"], app["after"] if b < 0 or a < 0: print(f"Error: Invalid consumption for {app['name']}") continue saved = b - a pct = (saved / b) * 100 if b > 0 else 0 total_before += b total_after += a status = "REDUCED" if saved > 0 else ("INCREASED" if saved < 0 else "NO CHANGE") print(f"{app['name']}: -{saved} kWh ({pct:.1f}%) [{status}]") net_savings = total_before - total_after overall_pct = (net_savings / total_before) * 100 goal_met = overall_pct >= target_saving_pct print("-" * 35) print(f"Total Before: {total_before} kWh | Total After: {total_after} kWh") print(f"Net Savings: {net_savings} kWh ({overall_pct:.2f}%)") print(f"Target Met (>={target_saving_pct}%): {'YES' if goal_met else 'NO'}")
```

FLOWCHART / PROCEDURE



PROGRAM OUTPUT

```
--- APPLIANCE ENERGY AUDIT --- HVAC Unit: -90 kWh (20.0%) [REDUCED] Lighting: -45 kWh (37.5%) [REDUCED] Refrigeration: -10 kWh (5.0%) [REDUCED] Water Heater: -55 kWh (30.6%) [REDUCED] ----------------------------------- Total Before: 950 kWh | Total After: 750 kWh Net Savings: 200 kWh (21.05%) Target Met (>=20.0%): YES
```

PROGRAM 1: Energy Conservation Before-and-After Analysis

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define data structures containing appliance names with their baseline (before) and post-intervention (after) energy consumption values in kWh, alongside the desired target conservation percentage.
3. Initialize cumulative counters `total_before` and `total_after` to zero.
4. Iterate through each appliance in the list:
    - Validate that neither before nor after consumption values are negative; display error if invalid.
    - Calculate absolute reduction: `saved = before - after`.
    - Calculate relative percentage reduction: `(saved / before) * 100`.
    - Add appliance values to `total_before` and `total_after`.
    - Display individual appliance energy variation and status.
5. Compute facility-wide net energy savings and overall percentage reduction.
6. Compare overall reduction percentage with target threshold to determine audit success.
7. Display consolidated energy statement, net units saved, and compliance status.
8. **Stop**.

B. PSEUDOCODE

```
START SET target_saving_pct = 20.0 SET total_before = 0, total_after = 0 INPUT appliances list of (name, before, after) FOR EACH app IN appliances DO IF app.before < 0 OR app.after < 0 THEN DISPLAY "Invalid reading for " + app.name CONTINUE END IF saved = app.before - app.after pct = (saved / app.before) * 100 total_before = total_before + app.before total_after = total_after + app.after IF saved > 0 THEN status = "REDUCED" ELSE IF saved < 0 THEN status = "INCREASED" ELSE status = "NO CHANGE" END IF DISPLAY app.name, saved, pct, status END FOR net_savings = total_before - total_after overall_pct = (net_savings / total_before) * 100 IF overall_pct >= target_saving_pct THEN goal_met = TRUE ELSE goal_met = FALSE END IF DISPLAY total_before, total_after, net_savings, overall_pct, goal_met STOP
```

2. Progressive Electricity Billing

Compute consumer electricity utility invoices using tiered consumption slabs, fixed charges, and tax levies.

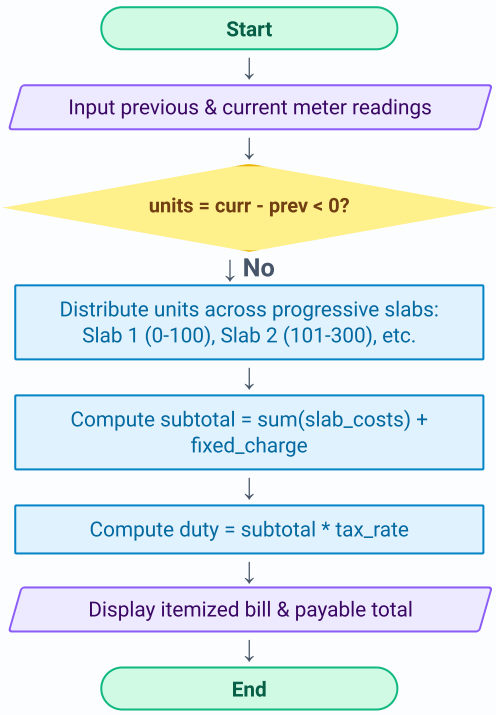
PROBLEM STATEMENT

> Design and implement a tiered slab-based utility billing engine for electricity consumption. The program takes meter readings, calculates units consumed, and computes energy charges progressively across defined slabs (e.g., 0–100 units at $1.50, 101–300 at $2.50, 301–500 at $4.00, and above 500 at $6.00). A base meter service charge and mandatory government surcharge (e.g., 5%) must be applied. The system must gracefully catch meter rollbacks or negative unit inputs.

PYTHON CODE

```
prev_reading = 12450 curr_reading = 12875
fixed_charge = 75.0 tax_rate = 0.05 units =
curr_reading - prev_reading if units < 0:
print("Error: Meter tampering or negative consumption
detected") else: remaining = units slab_charge = 0.0
slabs = [(100, 1.50), (200, 2.50), (200, 4.00),
(float('inf'), 6.00)] breakdown = [] for limit, rate
in slabs: if remaining <= 0: break billable =
min(remaining, limit) cost = billable * rate
slab_charge += cost breakdown.append((billable, rate,
cost)) remaining -= billable subtotal = slab_charge +
fixed_charge tax = subtotal * tax_rate total_bill =
subtotal + tax print(f"Units Consumed: {units} kWh")
for b, r, c in breakdown: print(f" {b} units @
${r:.2f} = ${c:.2f}") print(f"Energy Charge:
${slab_charge:.2f}") print(f"Fixed Service Fee:
${fixed_charge:.2f}") print(f"Electricity Duty (5%):
${tax:.2f}") print(f"Total Amount Payable:
${total_bill:.2f}")
```

FLOWCHART / PROCEDURE



PROGRAM OUTPUT

```
Units Consumed: 425 kWh 100 units @ $1.50 = $150.00 200 units @ $2.50 = $500.00 125 units @
$4.00 = $500.00 Energy Charge: $1150.00 Fixed Service Fee: $75.00 Electricity Duty (5%): $61.25
Total Amount Payable: $1286.25
```

PROGRAM 2: Progressive Electricity Billing

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Set tariff tier slabs with unit capacities and corresponding per-unit rates. Define fixed charge and regulatory tax percentage.
3. Input current meter reading and previous meter reading.
4. Calculate total units consumed by subtracting previous reading from current reading.
5. Validate that units consumed is greater than or equal to zero. If negative, flag invalid meter error and terminate.
6. Initialize remaining units to total units consumed and cumulative energy charge to zero.
7. For each slab in defined tariff schedule:
    - Determine units falling in current slab as minimum of remaining units and slab tier limit.
    - Calculate cost for this slab and accumulate into total energy charge.
    - Subtract billable units from remaining units; stop if remaining units reach zero.
8. Add fixed service charge to energy charge to obtain subtotal.
9. Calculate statutory duty/tax as percentage of subtotal and compute final gross bill.
10. Display detailed itemized slab breakdown, charges, taxes, and final payable amount.
11. **Stop**.

B. PSEUDOCODE

```
START SET fixed_charge = 75.0, tax_rate = 0.05 INPUT prev_reading, curr_reading
units = curr_reading - prev_reading IF units < 0 THEN DISPLAY "Error: Meter reading invalid
or rolled back" STOP END IF remaining = units slab_charge = 0.0 slabs = [(100, 1.50), (200,
2.50), (200, 4.00), (INFINITY, 6.00)] FOR EACH (limit, rate) IN slabs DO IF remaining <= 0
THEN EXIT FOR END IF billable = MIN(remaining, limit) cost = billable * rate slab_charge =
slab_charge + cost remaining = remaining - billable END FOR subtotal = slab_charge +
fixed_charge tax = subtotal * tax_rate total_bill = subtotal + tax DISPLAY units,
slab_charge, fixed_charge, tax, total_bill STOP
```

3. Progressive Water Tariff Analysis

Evaluate multi-tier conservation volumetric water tariffs with sewage disposal fees and category multipliers.

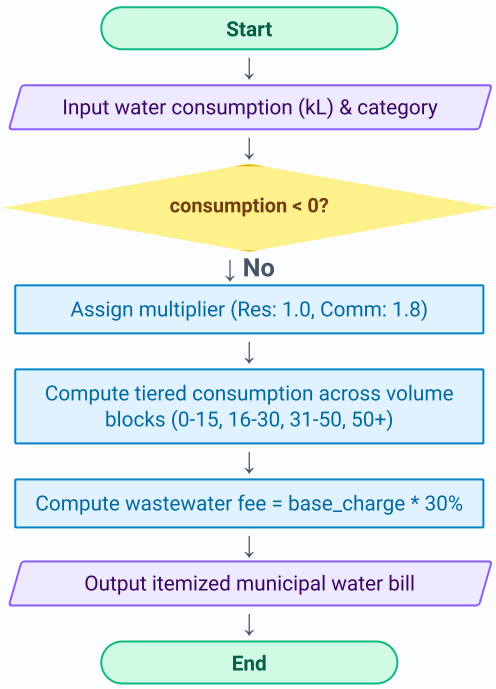
PROBLEM STATEMENT

> Municipal water utilities penalize excessive consumption through escalating volumetric tariff brackets. Construct a software model to process monthly water consumption in kilolitres (kL). Slabs are stratified (0–15 kL basic lifeline, 16–30 kL standard, 31–50 kL elevated, >50 kL commercial/luxury rate). Factor in customer category multipliers (Residential 1.0x, Commercial 1.8x) along with an environmental wastewater sewerage surcharge of 30% on consumption charges.

PYTHON CODE

```
consumption_kl = 38.0 category = "Residential"
# Options: Residential, Commercial cat_multiplier =
1.0 if category == "Residential" else 1.8 sewerage_pct
= 0.30 if consumption_kl < 0: print("Invalid
consumption value") else: slabs = [(15, 8.0), (15,
14.0), (20, 22.0), (float('inf'), 35.0)] rem =
consumption_kl base_charge = 0.0 for limit, rate in
slabs: if rem <= 0: break vol = min(rem, limit)
base_charge += vol * (rate * cat_multiplier) rem -=
vol sewerage_fee = base_charge * sewerage_pct
total_water_bill = base_charge + sewerage_fee
print(f"Customer Category: {category}")
print(f"Consumption: {consumption_kl:.1f} kL")
print(f"Water Volumetric Charge: ${base_charge:.2f}")
print(f"Sewerage & Environmental Fee:
${sewerage_fee:.2f}") print(f"Final Invoice Total:
${total_water_bill:.2f}")
```

FLOWCHART / PROCEDURE



PROGRAM OUTPUT

```
Customer Category: Residential Consumption: 38.0 kL Water Volumetric Charge: $506.00
Sewerage & Environmental Fee: $151.80 Final Invoice Total: $657.80
```

PROGRAM 3: Progressive Water Tariff Analysis

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define progressive slab volume thresholds (15 kL, 15 kL, 20 kL, unbounded) and standard rates.
3. Set category multipliers (1.0 for Residential, 1.8 for Commercial) and sewerage surcharge rate (0.30).
4. Input consumer water consumption in kilolitres (kL) and customer type.
5. Validate that input consumption is positive and customer type is recognized.
6. Set variable `remaining` to total consumption and `base_charge` to zero.
7. Iterate over each volumetric slab:
    - Determine consumed volume inside current slab as `MIN(remaining, slab_limit)`.
    - Compute slab cost applying unit rate adjusted by category multiplier.
    - Accumulate into `base_charge` and subtract billed volume from `remaining`.
    - Break when remaining volume is zero.
8. Compute sewerage and effluent treatment fee by multiplying base water charge by surcharge fraction.
9. Calculate grand total bill as sum of base volumetric charge and sewerage fee.
10. Display bill statement and category parameters.
11. **Stop**.

B. PSEUDOCODE

```
START INPUT consumption_kl, category IF consumption_kl < 0 THEN DISPLAY "Invalid consumption quantity" STOP END IF IF category == "Residential" THEN multiplier = 1.0 ELSE multiplier = 1.8 END IF rem = consumption_kl base_charge = 0.0 slabs = [(15, 8.0), (15, 14.0), (20, 22.0), (INFINITY, 35.0)] FOR EACH (limit, rate) IN slabs DO IF rem <= 0 THEN EXIT FOR END IF vol = MIN(rem, limit) base_charge = base_charge + (vol * rate * multiplier) rem = rem - vol END FOR sewerage_fee = base_charge * 0.30 total_bill = base_charge + sewerage_fee DISPLAY category, consumption_kl, base_charge, sewerage_fee, total_bill STOP
```

4. Repeated Word and Phrase Detection

Tokenize text corpora, eliminate stop words, identify n-gram phrases, and highlight redundant linguistic patterns.

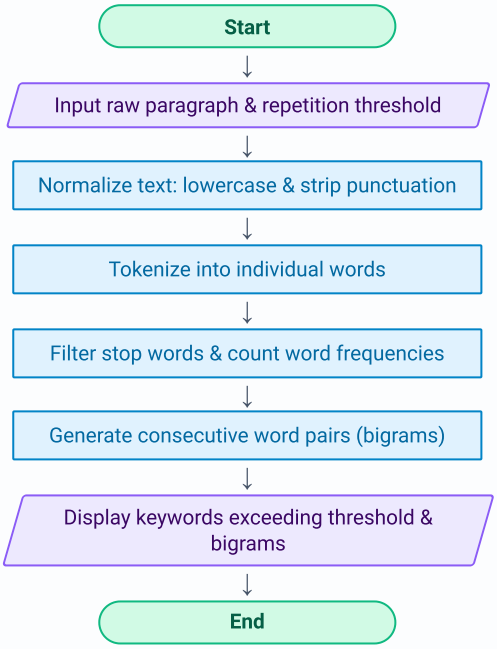
PROBLEM STATEMENT

> In automated essay analysis and editorial proofing, detecting excessive word repetition and repetitive n-gram phrase patterns is essential. Develop a Python program to clean an input paragraph (case-folding, punctuation stripping), filter out grammatical noise (common stop words), tally unique word frequencies, extract bigram collocations (two-word phrases), and flag terms exceeding a specified threshold of repetition.

PYTHON CODE

```
import string from collections import Counter text = "Data science requires data cleaning. Good data produces good model outcomes. Data is key." stop_words = {"is", "the", "in", "and", "a", "an", "to"} threshold = 2 # Clean and tokenize clean_chars = [c.lower() if c not in string.punctuation else " " for c in text] words = "".join(clean_chars).split() meaningful_words = [w for w in words if w not in stop_words] # Frequency analysis word_counts = Counter(meaningful_words) repeated_words = {w: c for w, c in word_counts.items() if c >= threshold} # Bigram extraction bigrams = [f"{words[i]} {words[i+1]}" for i in range(len(words)-1)] bigram_counts = Counter(bigrams) print("Repeated Keywords (Frequency >= 2):") for w, cnt in repeated_words.items(): print(f" '{w}': {cnt} times") print("\nIdentified Phrase Bigrams:") for phrase, cnt in bigram_counts.most_common(3): print(f" '{phrase}': {cnt} occurrences")
```

FLOWCHART / PROCEDURE



PROGRAM OUTPUT

```
Repeated Keywords (Frequency >= 2): 'data': 3 times 'good': 2 times Identified Phrase Bigrams: 'data science': 1 occurrences 'science requires': 1 occurrences 'requires data': 1 occurrences
```

PROGRAM 4: Repeated Word and Phrase Detection

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define input text string and initialize a set of non-informative stop words (articles, prepositions, auxiliary verbs). Set repetition threshold limit.
3. Strip all punctuation characters from input string and convert entire text to lowercase characters.
4. Split sanitized string by whitespace into an ordered list of word tokens.
5. Filter list of tokens by omitting terms present in stop words collection.
6. Tally occurrences of each unique word in filtered token list.
7. Identify all words whose frequency count equals or exceeds specified threshold.
8. Iterate through original adjacent token pairs to construct 2-word phrase n-grams (bigrams) and compute frequencies.
9. Display detected redundant words along with their counts, followed by top repeated phrases.
10. **Stop**.

B. PSEUDOCODE

```
      START INPUT text, threshold SET stop_words = ["is", "the", "in", "and", "a", "an",
"to"] clean_text = REMOVE_PUNCTUATION(LOWERCASE(text)) words = SPLIT(clean_text, " ")
meaningful_words = [] FOR EACH w IN words DO IF w NOT IN stop_words AND LENGTH(w) > 0 THEN
APPEND w TO meaningful_words END IF END FOR word_freq = MAP() FOR EACH w IN
meaningful_words DO word_freq[w] = word_freq.GET(w, 0) + 1 END FOR bigram_freq = MAP() FOR
i = 0 TO LENGTH(words) - 2 DO phrase = words[i] + " " + words[i+1] bigram_freq[phrase] =
bigram_freq.GET(phrase, 0) + 1 END FOR DISPLAY "Words meeting threshold:" FOR EACH (w,
count) IN word_freq DO IF count >= threshold THEN DISPLAY w, count END IF END FOR DISPLAY
"Frequent Bigrams:", bigram_freq STOP
```

5. Sports Tournament Ranking System

Compute league standings, apply match points (Win/Draw/Loss), and resolve tie-breakers via goal differentials.

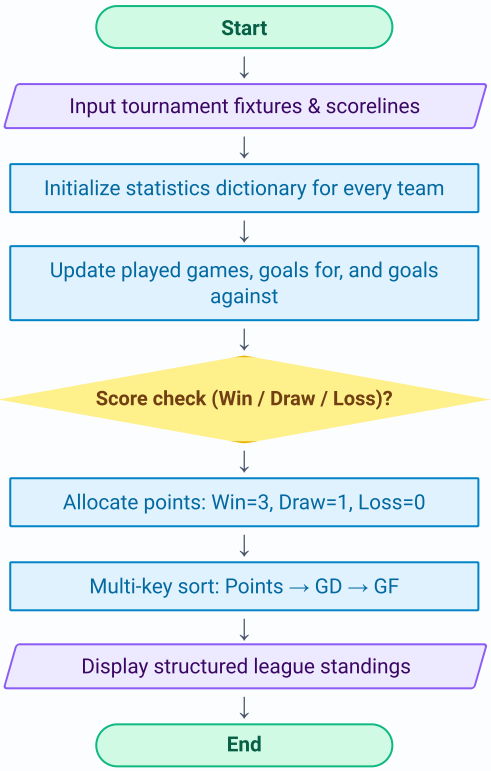
PROBLEM STATEMENT

> Construct a sports league standings processor. Given a schedule of completed fixtures with scores, compute total matches played, wins, draws, losses, goals scored, and goals conceded for each club. Points are awarded as Win=3, Draw=1, Loss=0. Teams must be sorted and ranked by total points, then by goal differential (scored - conceded), then by total goals scored, and finally alphabetically to break ties cleanly.

PYTHON CODE

FLOWCHART / PROCEDURE

```
matches = [ ("Hawks", 2, "Eagles", 1), ("Falcons", 0, "Hawks", 0), ("Eagles", 3, "Falcons", 1), ("Eagles", 2, "Hawks", 2) ] teams = {} def init_team(name): if name not in teams: teams[name] = {"P": 0, "W": 0, "D": 0, "L": 0, "GF": 0, "GA": 0, "Pts": 0} for t1, s1, t2, s2 in matches: init_team(t1); init_team(t2) teams[t1]["P"] += 1; teams[t2]["P"] += 1 teams[t1]["GF"] += s1; teams[t1]["GA"] += s2 teams[t2]["GF"] += s2; teams[t2]["GA"] += s1 if s1 > s2: teams[t1]["W"] += 1; teams[t1]["Pts"] += 3 teams[t2]["L"] += 1 elif s2 > s1: teams[t2]["W"] += 1; teams[t2]["Pts"] += 3 teams[t1]["L"] += 1 else: teams[t1]["D"] += 1; teams[t1]["Pts"] += 1 teams[t2]["D"] += 1; teams[t2]["Pts"] += 1 leaderboard = sorted(teams.items(), key=lambda x: (x[1]["Pts"], x[1]["GF"] - x[1]["GA"], x[1]["GF"]), reverse=True) print("Pos Team P W D L GF GA GD Pts") for r, (t, s) in enumerate(leaderboard, 1): gd = s["GF"] - s["GA"] print(f"{r:<4} {t:<8} {s['P']} {s['W']} {s['D']} {s['L']} {s['GF']} {s['GA']} {gd:+d} {s['Pts']}")
```



PROGRAM OUTPUT

```
Pos Team P W D L GF GA GD Pts 1 Hawks 3 1 2 0 4 3 +1 5 2 Eagles 3 1 1 1 6 5 +1 4 3 Falcons 2 0 1 1 1 3 -2 1
```

PROGRAM 5: Sports Tournament Ranking System

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Initialize an empty collection `teams` to store statistics (matches played, won, drawn, lost, goals scored, goals conceded, points).
3. Input the sequence of played match fixtures consisting of Team 1, Score 1, Team 2, and Score 2.
4. For each match record:
    - Ensure both participating teams exist in the statistics dictionary.
    - Increment matches played counter for both teams.
    - Add Score 1 to Team 1's goals for and Team 2's goals against.
    - Add Score 2 to Team 2's goals for and Team 1's goals against.
    - If Score 1 > Score 2: award 3 points and increment wins for Team 1; increment losses for Team 2.
    - If Score 2 > Score 1: award 3 points and increment wins for Team 2; increment losses for Team 1.
    - If Score 1 equals Score 2: award 1 point and increment draws for both teams.
5. Calculate goal difference (`GD = GF - GA`) for each team.
6. Sort all teams using compound ranking priorities: Points (descending), GD (descending), and GF (descending).
7. Format and display the ranked tournament leaderboard table.
8. **Stop**.

B. PSEUDOCODE

```
START INPUT matches list of (team1, score1, team2, score2) SET teams = MAP() FOR EACH (t1, s1, t2, s2) IN matches DO INITIALIZE t1, t2 IN teams IF NOT PRESENT teams[t1].played += 1 teams[t2].played += 1 teams[t1].gf += s1; teams[t1].ga += s2 teams[t2].gf += s2; teams[t2].ga += s1 IF s1 > s2 THEN teams[t1].wins += 1; teams[t1].pts += 3 teams[t2].losses += 1 ELSE IF s2 > s1 THEN teams[t2].wins += 1; teams[t2].pts += 3 teams[t1].losses += 1 ELSE teams[t1].draws += 1; teams[t1].pts += 1 teams[t2].draws += 1; teams[t2].pts += 1 END IF END FOR FOR EACH team IN teams DO team.gd = team.gf - team.ga END FOR SORT teams BY (pts DESC, gd DESC, gf DESC) DISPLAY standings table with ranks, records, points STOP
```

6. Hospital Inpatient Billing System

Audit room occupancy, physician consultations, pharmacy & lab expenses, insurance coverage, and co-pays.

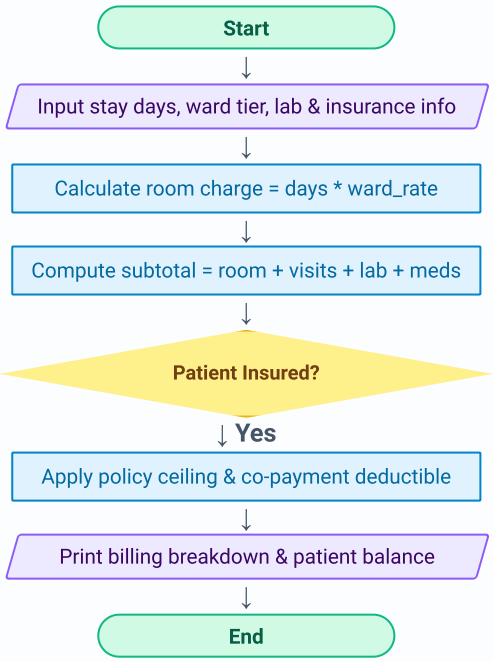
PROBLEM STATEMENT

Implement an inpatient billing management module for a hospital healthcare facility. Calculate room charges based on ward categories (General: $50/day, Semi-Private: $100/day, ICU: $300/day) multiplied by duration of stay. Aggregate medical services including specialist doctor visits, lab tests, and medication charges. If the patient has insurance, compute coverage up to the policy limit after deducting deductible co-payments.

PYTHON CODE

```
ward_rates = {"General": 50, "Semi-Private": 100, "ICU": 300} patient = { "name": "Sarah Jenkins", "ward": "Semi-Private", "days": 5, "consultations": 4, "consult_rate": 80, "lab_charges": 350.0, "pharmacy": 220.0, "insured": True, "policy_limit": 1000.0, "copay_pct": 0.20 } # Room and procedure charges room_cost = patient["days"] * ward_rates[patient["ward"]] doctor_fee = patient["consultations"] * patient["consult_rate"] gross_total = room_cost + doctor_fee + patient["lab_charges"] + patient["pharmacy"] if patient["insured"]: eligible_claim = min(gross_total, patient["policy_limit"]) insurance_paid = eligible_claim * (1.0 - patient["copay_pct"]) patient_payable = (gross_total - eligible_claim) + (eligible_claim * patient["copay_pct"]) else: insurance_paid = 0.0 patient_payable = gross_total print(f"Patient Name: {patient['name']}") print(f"Room Charge ({patient['days']} days in {patient['ward']}): ${room_cost:.2f}") print(f"Doctor Visits: ${doctor_fee:.2f} | Lab: ${patient['lab_charges']:.2f}") print(f"Pharmacy Medicines: ${patient['pharmacy']:.2f}") print(f"Gross Hospital Bill: ${gross_total:.2f}") print(f"Insurance Claim Settled: ${insurance_paid:.2f}") print(f"Net Amount Payable by Patient: ${patient_payable:.2f}")
```

FLOWCHART / PROCEDURE



PROGRAM OUTPUT

```
Patient Name: Sarah Jenkins Room Charge (5 days in Semi-Private): $500.00 Doctor Visits: $320.00 | Lab: $350.00 Pharmacy Medicines: $220.00 Gross Hospital Bill: $1390.00 Insurance Claim Settled: $800.00 Net Amount Payable by Patient: $590.00
```

PROGRAM 6: Hospital Inpatient Billing System

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define lookup tables for daily ward accommodation rates (General, Semi-Private, ICU).
3. Input patient admission records: admitted ward category, length of stay in days, doctor consultation count, consultation rate, laboratory test charges, and medication total.
4. Input insurance details: policy validity status, maximum claim limit, and patient co-pay percentage.
5. Calculate total room charges by multiplying days of stay by daily rate for designated ward category.
6. Calculate total medical practitioner fees by multiplying consultations by consultation rate.
7. Compute `gross_total` by summing room charges, practitioner fees, lab fees, and medicine expenses.
8. Evaluate insurance coverage:
    - If patient is insured, determine eligible coverage as the minimum of `gross_total` and `policy_limit`.
    - Calculate amount absorbed by insurance company after deducting patient's co-pay portion.
    - Compute net out-of-pocket cost payable by patient (uncovered portion + co-pay share).
    - If uninsured, patient bears 100% of gross total.
9. Display detailed itemized hospital statement and patient payment obligation.
10. **Stop**.

B. PSEUDOCODE

```
START SET ward_rates = {"General": 50, "Semi-Private": 100, "ICU": 300} INPUT ward_type, days, consults, consult_rate, lab_cost, pharma_cost INPUT is_insured, policy_limit, copay_pct room_charge = days * ward_rates[ward_type] doctor_charge = consults * consult_rate gross_bill = room_charge + doctor_charge + lab_cost + pharma_cost IF is_insured == TRUE THEN covered_base = MIN(gross_bill, policy_limit) insurance_paid = covered_base * (1.0 - copay_pct) patient_due = (gross_bill - covered_base) + (covered_base * copay_pct) ELSE insurance_paid = 0.0 patient_due = gross_bill END IF DISPLAY room_charge, doctor_charge, lab_cost, pharma_cost DISPLAY gross_bill, insurance_paid, patient_due STOP
```

7. Multi-Product Shopping Budget Constraint

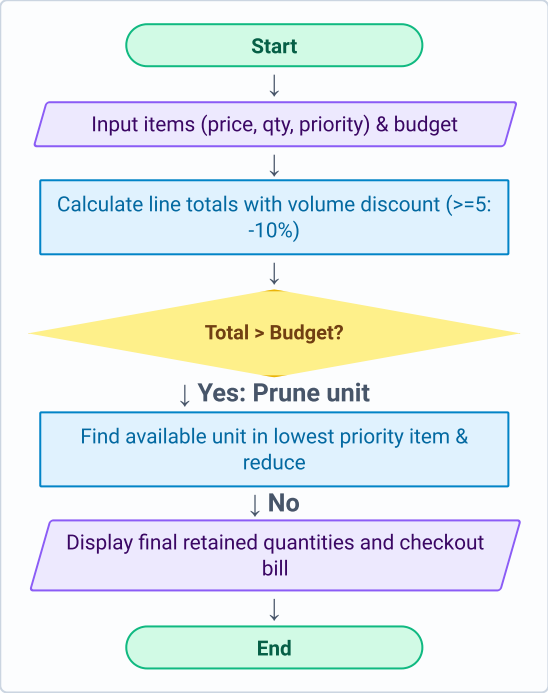
Enforce budget ceiling through automated reduction of non-essential items based on priority order.

PROBLEM STATEMENT

> Build a shopping basket optimizer that applies quantity pricing and volume discounts (e.g., 10% discount on item lines where quantity >= 5). If the initial checkout total exceeds a pre-assigned spending budget limit, the algorithm must iteratively prune or decrement product quantities following a defined low-to-high priority ranking until total cost falls strictly within budget, reporting removed units and remaining contents.

PYTHON CODE

```
budget = 2500.0 basket = [ {"name": "Notebook", "price": 100.0, "qty": 8, "priority": 3}, {"name": "Pen pack", "price": 80.0, "qty": 10, "priority": 2}, {"name": "Backpack", "price": 900.0, "qty": 2, "priority": 1} ] def compute_total(items): tot = 0.0 for it in items: line = it["price"] * it["qty"] disc = 0.10 * line if it["qty"] >= 5 else 0.0 tot += (line - disc) return tot initial_tot = compute_total(basket) curr_tot = initial_tot # Sort by lowest priority first (priority 1 = least essential) basket.sort(key=lambda x: x["priority"]) while curr_tot > budget: reduced = False for it in basket: if it["qty"] > 0: it["qty"] -= 1 reduced = True break if not reduced: break curr_tot = compute_total(basket) print(f"Initial Cart Value: ${initial_tot:.2f}") print(f"Budget Limit: ${budget:.2f}") print("Final Optimized Basket:") for it in basket: print(f" {it['name']}: {it['qty']} units @ ${it['price']:.2f}") print(f"Final Cart Total: ${curr_tot:.2f} (Within Budget: {curr_tot <= budget})")
```



PROGRAM OUTPUT

```
Initial Cart Value: $3060.00 Budget Limit: $2500.00 Final Optimized Basket: Backpack: 1 units @ $900.00 Pen pack: 10 units @ $80.00 Notebook: 8 units @ $100.00 Final Cart Total: $2340.00 (Within Budget: True)
```

PROGRAM 7: Multi-Product Shopping Budget Constraint

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Input allowable customer budget limit and shopping basket containing items with attributes: product name, unit price, requested quantity, and priority rank.
3. Define a helper function to compute gross basket cost by evaluating each line item and applying a 10% volume discount when quantity is 5 or more.
4. Calculate initial total basket valuation.
5. If initial cost does not exceed budget, proceed directly to summary output.
6. Otherwise, order basket items in ascending order of importance/priority (least essential first).
7. While total expenditure strictly exceeds budget and at least one item has quantity greater than zero:
    - Select the lowest-priority item possessing quantity > 0.
    - Decrement its quantity by 1 unit.
    - Recompute current cart expenditure using discount rules.
8. Display initial checkout value, units pruned, final cart contents, and final reconciled total.
9. **Stop**.

B. PSEUDOCODE

```
       START INPUT budget, items list of (name, price, qty, priority) FUNCTION
GET_TOTAL(items) tot = 0.0 FOR EACH it IN items DO line = it.price * it.qty IF it.qty >= 5
THEN line = line * 0.90 END IF tot = tot + line END FOR RETURN tot END FUNCTION
initial_total = GET_TOTAL(items) SORT items BY priority ASCENDING WHILE GET_TOTAL(items) >
budget DO reduced = FALSE FOR EACH it IN items DO IF it.qty > 0 THEN it.qty = it.qty - 1
reduced = TRUE EXIT FOR END IF END FOR IF reduced == FALSE THEN EXIT WHILE END IF END WHILE
final_total = GET_TOTAL(items) DISPLAY initial_total, items, final_total STOP
```

8. Multi-Stage Scholarship Eligibility Engine

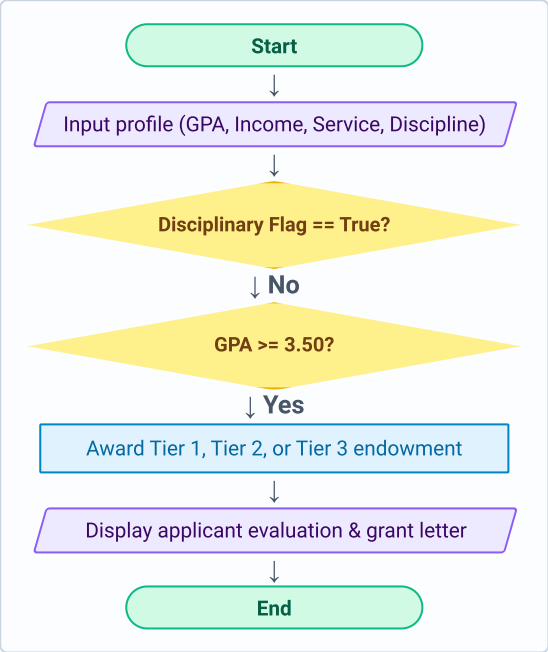
Filter academic awards via GPA benchmarks, family income thresholds, extracurricular points, and disciplinary clean records.

PROBLEM STATEMENT

> Higher educational institutions award multi-tier scholarship endowments based on holistic screening. Implement an evaluation engine that screens student applicants through progressive hurdle criteria: Grade Point Average (minimum 3.5/4.0), household income threshold, involvement index (sports/community service), and zero disciplinary flags. Successful applicants are classified into Tier 1 (Full Ride), Tier 2 (Half Tuition), or Tier 3 (Grant).

PYTHON CODE

```
applicants = [ {"id": "SCH-101", "name": "Elena Rostova", "gpa": 3.92, "income": 42000, "service_hrs": 60, "disciplinary": False}, {"id": "SCH-102", "name": "Marcus Vance", "gpa": 3.65, "income": 75000, "service_hrs": 20, "disciplinary": False}, {"id": "SCH-103", "name": "Amina Yusuf", "gpa": 3.85, "income": 28000, "service_hrs": 80, "disciplinary": False}, {"id": "SCH-104", "name": "David Cole", "gpa": 3.40, "income": 35000, "service_hrs": 50, "disciplinary": True} ] results = [] for s in applicants: if s["disciplinary"]: status, award = "REJECTED", "Disciplinary Violation" elif s["gpa"] < 3.50: status, award = "REJECTED", "GPA Below Threshold (<3.50)" else: # Eligible - evaluate scholarship award tiers if s["gpa"] >= 3.85 and s["income"] <= 50000 and s["service_hrs"] >= 50: award = "Tier 1: Full-Ride Tuition Scholarship" elif s["gpa"] >= 3.60 and s["income"] <= 60000: award = "Tier 2: 50% Tuition Grant" else: award = "Tier 3: Merit Commendation Book Stipend" status = "AWARDED" results.append((s["id"], s["name"], status, award)) print(f"{'ID':<8} {'Name':<15} {'Status':<10} {'Award Allocation'}") print("-" * 65) for sid, name, st, aw in results: print(f"{sid:<8} {name:<15} {st:<10} {aw}")
```



PROGRAM OUTPUT

```
ID Name Status Award Allocation ----------------------------------------------------------- SCH-101 Elena Rostova AWARDED Tier 1: Full-Ride Tuition Scholarship SCH-102 Marcus Vance AWARDED Tier 3: Merit Commendation Book Stipend SCH-103 Amina Yusuf AWARDED Tier 1: Full-Ride Tuition Scholarship SCH-104 David Cole REJECTED Disciplinary Violation
```

PROGRAM 8: Multi-Stage Scholarship Eligibility Engine

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define institutional qualification gates: minimum GPA (3.50), clean disciplinary standing flag, income brackets, and community service hours.
3. Input candidate record set consisting of Candidate ID, Full Name, cumulative GPA, annual family income, community service hours, and disciplinary record boolean flag.
4. For each student candidate record in candidate set:
    - Examine disciplinary record. If flag is True, reject application immediately.
    - Examine GPA. If GPA is below 3.50 minimum standard, reject application for academic shortfall.
    - If candidate meets all primary gates, evaluate for award tiers:
        - If GPA >= 3.85, income <= $50,000, and service hours >= 50, allocate **Tier 1 Full-Ride**.
        - Otherwise, if GPA >= 3.60 and income <= $60,000, allocate **Tier 2 Tuition Grant**.
        - Otherwise, allocate **Tier 3 Merit Stipend**.
5. Format and generate comprehensive scholarship allocation decisions roster.
6. **Stop**.

B. PSEUDOCODE

```
       START INPUT applicants list of (id, name, gpa, income, service_hrs, disciplinary)
FOR EACH s IN applicants DO IF s.disciplinary == TRUE THEN status = "REJECTED" award =
"Disciplinary Record Disqualification" ELSE IF s.gpa < 3.50 THEN status = "REJECTED" award
= "GPA Below Minimum 3.50 Threshold" ELSE status = "AWARDED" IF s.gpa >= 3.85 AND s.income
<= 50000 AND s.service_hrs >= 50 THEN award = "Tier 1: Full-Ride Tuition Scholarship" ELSE
IF s.gpa >= 3.60 AND s.income <= 60000 THEN award = "Tier 2: 50% Tuition Grant" ELSE award
= "Tier 3: Merit Commendation Book Stipend" END IF END IF DISPLAY s.id, s.name, status,
award END FOR STOP
```

9. Student Attendance Recovery Computation

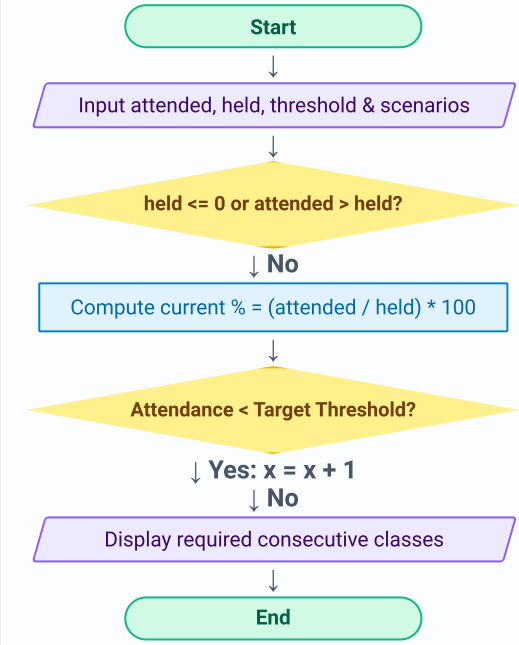
Compute attendance percentage and determine the minimum consecutive classes needed to meet an eligibility threshold.

PROBLEM STATEMENT

> Given classes attended and total classes held, compute current attendance percentage. For each future scenario of additional classes scheduled, determine the smallest number of consecutive classes the student must attend to reach the mandated eligibility threshold (e.g. 75%). Validate inputs and flag impossible recovery states or data anomalies.

PYTHON CODE

```
       attended = 36 held = 50 threshold = 75.0
future_scenarios = [0, 5, 10] # extra classes held in
future if held <= 0 or attended < 0 or attended >
held: print("Invalid attendance data") elif not (0 <
threshold <= 100): print("Invalid threshold") else:
current = (attended / held) * 100 print(f"Current
Attendance: {current:.2f}%") for extra_held in
future_scenarios: # Determine minimum consecutive
classes 'x' to attend x = 0 while ((attended + x) /
(held + extra_held + x)) * 100 < threshold: x += 1 if
x > 10000: # safety break break print(f"Scenario +
{extra_held} held: attend {x} more consecutive
classes")
```



PROGRAM OUTPUT

```
      Current Attendance: 72.00% Scenario +0 held: attend 6 more consecutive classes Scenario +5
held: attend 21 more consecutive classes Scenario +10 held: attend 36 more consecutive classes
```

PROGRAM 9: Student Attendance Recovery Computation

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Input total classes attended, total classes conducted to date, and target eligibility percentage threshold.
3. Validate that classes held is positive, attended count is between 0 and held count, and target threshold is within (0, 100].
4. Calculate current attendance ratio: `(attended / held) * 100` and display percentage.
5. For each future scenario representing additional classes scheduled:
    - Set required additional attended classes variable `x = 0`.
    - While the projected ratio `((attended + x) / (held + extra_held + x)) * 100` remains strictly below required threshold, increment x by 1.
    - Report the minimal non-negative integer value of x.
6. Display detailed recovery roadmap for each prospective course scheduling scenario.
7. **Stop**.

B. PSEUDOCODE

```
START INPUT attended, held, threshold, future_scenarios IF held <= 0 OR attended < 0 OR attended > held THEN DISPLAY "Invalid attendance data" STOP END IF IF threshold <= 0 OR threshold > 100 THEN DISPLAY "Invalid threshold" STOP END IF current = (attended / held) * 100 DISPLAY current FOR EACH extra_held IN future_scenarios DO x = 0 WHILE ((attended + x) / (held + extra_held + x)) * 100 < threshold DO x = x + 1 IF x > 10000 THEN EXIT WHILE END IF END WHILE DISPLAY extra_held, x END FOR STOP
```

10. Valid Books and ISBN Inventory Verification

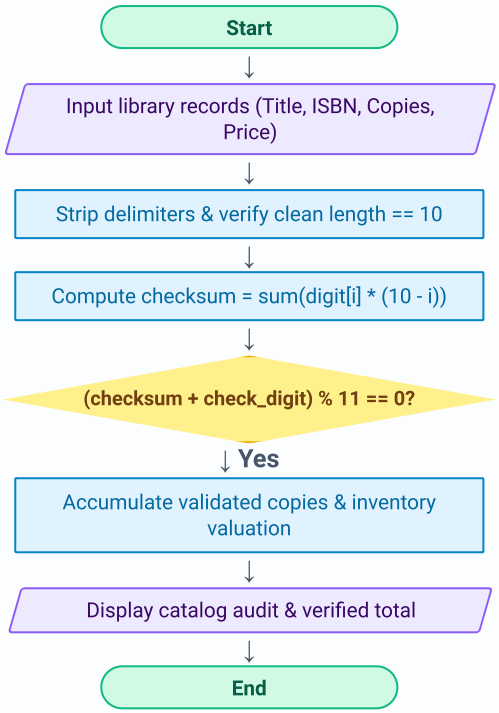
Verify ISBN-10 check-digits via modular checksum algorithms and manage catalog circulation stock.

PROBLEM STATEMENT

> Library information systems enforce bibliographic integrity by verifying International Standard Book Numbers (ISBN). Develop a catalog verification module to process accession records with ISBN-10 codes. The program must strip hyphens, validate length, process the weighted checksum formula `sum(digit * (10 - i)) mod 11 == 0` (supporting 'X' as 10), reject malformed codes, and compute available circulation inventory valuation.

PYTHON CODE

```
catalog = [ {"title": "Clean Code", "isbn": "0-13-235088-2", "copies": 4, "price": 45.0}, {"title": "Design Patterns", "isbn": "0-201-63361-2", "copies": 2, "price": 55.0}, {"title": "Faulty Volume", "isbn": "0-13-999999-9", "copies": 1, "price": 30.0}, {"title": "Algorithms", "isbn": "0-262-03384-4", "copies": 3, "price": 85.0} ] def is_valid_isbn10(isbn_str): raw = isbn_str.replace("-", "").strip() if len(raw) != 10: return False total = 0 for i in range(9): if not raw[i].isdigit(): return False total += int(raw[i]) * (10 - i) check = 10 if raw[9].upper() == 'X' else (int(raw[9]) if raw[9].isdigit() else -1) if check == -1: return False return (total + check) % 11 == 0 valid_stock, total_value = 0, 0.0 print("--- LIBRARY CATALOG AUDIT ---") for b in catalog: valid = is_valid_isbn10(b["isbn"]) status = "VALID" if valid else "CORRUPTED ISBN" print(f"'{b['title']}' [{b['isbn']}]: {status}") if valid: valid_stock += b["copies"] total_value += b["copies"] * b["price"] print("-" * 35) print(f"Verified Book Titles in Stock: {valid_stock} units") print(f"Total Verified Inventory Value: ${total_value:.2f}")
```



PROGRAM OUTPUT

```
--- LIBRARY CATALOG AUDIT --- 'Clean Code' [0-13-235088-2]: VALID 'Design Patterns' [0-201-63361-2]: VALID 'Faulty Volume' [0-13-999999-9]: CORRUPTED ISBN 'Algorithms' [0-262-03384-4]: VALID ---------------------------------- Verified Book Titles in Stock: 9 units Total Verified Inventory Value: $545.00
```

PROGRAM 10: Valid Books and ISBN Inventory Verification

Algorithmic formulation and formal pseudocode specification.

A. ALGORITHM

1. **Start**.
2. Define catalog database entries containing book title, ISBN-10 identification string, available stock copies, and unit purchase cost.
3. Initialize cumulative counters for validated physical books and total verified inventory value.
4. For each book record in catalog:
    - Clean the ISBN string by removing hyphen separators and extraneous whitespace.
    - Validate that sanitized string consists of exactly 10 characters.
    - Check that characters 1 through 9 are decimal digits; multiply each digit by its decreasing positional weight from 10 down to 2 and accumulate into sum.
    - Evaluate final check character: if character is 'X' or 'x', check value is 10; otherwise if numeric, convert to its digit value.
    - If total weighted sum plus check value modulo 11 equals 0, authenticate book ISBN as valid.
    - If authentic, add copies to verified stock count and augment inventory valuation by `copies * unit_price`. Otherwise, flag corrupted code.
5. Display verification log, verified inventory units, and asset valuation.
6. **Stop**.

B. PSEUDOCODE

```
       START INPUT catalog list of (title, isbn, copies, price) SET valid_stock = 0,
total_value = 0.0 FUNCTION IS_ISBN10_VALID(isbn_str) raw = REPLACE(isbn_str, "-", "") IF
LENGTH(raw) != 10 THEN RETURN FALSE sum = 0 FOR i = 0 TO 8 DO IF NOT IS_DIGIT(raw[i]) THEN
RETURN FALSE sum = sum + TO_INT(raw[i]) * (10 - i) END FOR IF raw[9] == 'X' OR raw[9] ==
'x' THEN check = 10 ELSE IF IS_DIGIT(raw[9]) THEN check = TO_INT(raw[9]) ELSE RETURN FALSE
END IF RETURN (sum + check) MOD 11 == 0 END FUNCTION FOR EACH b IN catalog DO IF
IS_ISBN10_VALID(b.isbn) THEN valid_stock = valid_stock + b.copies total_value = total_value
+ (b.copies * b.price) DISPLAY b.title, b.isbn, "VALID" ELSE DISPLAY b.title, b.isbn,
"CORRUPTED ISBN" END IF END FOR DISPLAY valid_stock, total_value STOP
```